

A HANDS-ON, CODE-FIRST COURSE

LangChain From Zero

Build real GenAI apps with Gemini & local models

PHASE 3 — PROMPTS & OUTPUT PARSERS

Prerequisites: Phase 2 done — comfortable with messages, `invoke`, `stream`, `batch`, and parameters.

Targets: ChatPromptTemplate · MessagesPlaceholder · few-shot · StrOutputParser · structured output (Pydantic)

Estimated time for this phase: 75–110 minutes

Where this phase sits

You can talk to a model. Now you'll stop hard-coding prompt strings and start building **reusable templates** — including few-shot examples — then force the model to hand back **clean, structured data** instead of free-form prose.

Ph 0 ✓ **Environment Setup**

Ph 1 ✓ **LangChain Fundamentals**

Ph 2 ✓ **Working with Models** — messages, invoke/stream/batch, parameters.

Ph 3 **Prompts & Output Parsers** ← you are here
Prompt templates, message placeholders, few-shot prompting, structured output with Pydantic.

Ph 4 **Chains & LCEL Composition** — parallel, routing, custom functions, debugging.

Ph 5–8 **Memory · Documents · Embeddings · RAG**

Ph 9–12 **Tools & Agents · LangGraph · Deployment · Capstone**

Phase 3 · Prompts & Output Parsers

WHAT YOU'LL BE ABLE TO DO AFTER THIS PHASE

Build reusable `ChatPromptTemplate` s with roles and variables; slot a dynamic list of past messages in with `MessagesPlaceholder` ; teach a model by example using few-shot prompting; and guarantee machine-readable output with Pydantic-based **structured output** — on both Gemini and Ollama.

In this phase

1. 3.1 — Why templates? The trouble with f-strings
2. 3.2 — `ChatPromptTemplate` : roles + variables
3. 3.3 — `MessagesPlaceholder` : a slot for past messages
4. 3.4 — Few-shot prompting: teaching by example
5. 3.5 — Output parsers & the problem with prose
6. 3.6 — Structured output with Pydantic (the modern way)
7. 3.7 — Common errors & troubleshooting
8. 3.8 — Mini practice exercise
9. Terms introduced this phase

3.1 — Why templates? The trouble with f-strings

You could build prompts with plain Python f-strings: `f"Summarize {text}"`. For one quick call that's fine. But real apps reuse the same prompt with different inputs, mix in a system persona, insert conversation history, and get validated before they run. f-strings give you none of that. A **prompt template** does.

TERM: PROMPT TEMPLATE

A **prompt template** is a reusable prompt with labelled `{blanks}`. You define it once, then fill the blanks with a dict at call time. It's a Runnable (Phase 1): input a dict → output a ready-to-send prompt. Because it's a Runnable, it pipes straight into a model.

3.2 — `ChatPromptTemplate` : roles + variables

Since we use chat models (Phase 2), we use `ChatPromptTemplate` — it builds a *list of role-tagged messages*, not just a string. The cleanest constructor is `from_messages`, which takes `(role, text)` tuples:

```
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
```

```

load_dotenv()

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a tutor who explains things to a {age}-year-old."), # 1
    ("human", "Explain {concept}."), # 2
]) # 3

model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")
chain = prompt | model | StrOutputParser() # 4

print(chain.invoke({"age": 8, "concept": "rainbows"})) # 5

```

Line	Explanation
1	<code>from_messages</code> takes a list of messages. Each is a <code>(role, text)</code> tuple — far less typing than building <code>SystemMessage(...)</code> objects by hand.
2–3	Roles <code>"system"</code> and <code>"human"</code> (also <code>"ai"</code>). The <code>{age}</code> and <code>{concept}</code> are named blanks. Note you can have variables in <i>any</i> message, including the system one.
4	The template is a Runnable, so it pipes straight into the model and parser — the Phase 1 pattern, now with a richer prompt.
5	<code>invoke</code> with a dict whose keys match every blank: <code>age</code> and <code>concept</code> . The template fills them, produces two messages, and sends them on.

Expected output:

```

A rainbow is like nature's painting in the sky! When sunlight shines through
tiny raindrops, the light bends and splits into all its hidden colors...

```

WHAT THE TEMPLATE ACTUALLY PRODUCES

Calling `prompt.invoke({"age": 8, "concept": "rainbows"})` on its own returns a `ChatPromptValue` wrapping:

```

[ SystemMessage("You are a tutor who explains things to a 8-year-old."),
  HumanMessage("Explain rainbows.") ]

```

Exactly the message list from Phase 2 — built for you. (For a single-message prompt with no roles, `ChatPromptTemplate.from_template("Explain {concept}.")` is the shorthand you saw in Phase 1.)

As always, the Ollama version changes only the model line: `model = ChatOllama(model="llama3.2")` (and drop `load_dotenv()`).

3.3 — `MessagesPlaceholder` : a slot for past messages

Templates so far have fixed slots for *text*. But to make a chatbot remember earlier turns, you need to drop in a whole *list of previous messages* whose length you don't know ahead of time. That's what

`MessagesPlaceholder` is for — a placeholder that gets filled with a list of messages.

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    MessagesPlaceholder("history", # 1 a slot named "history"
    ("human", "{question}"),
])

filled = prompt.invoke({ # 2
    "history": [ # 3 a list of earlier messages
        HumanMessage("My name is Sam."),
        AIMessage("Nice to meet you, Sam!"),
    ],
    "question": "What's my name?",
})
print(filled.messages)
```

Line	Explanation
1	<code>MessagesPlaceholder("history")</code> reserves a spot that will be filled by a <i>list</i> of messages under the key <code>"history"</code> . (Shorthand: the tuple <code>("placeholder", "{history}")</code> does the same.)
2–3	At call time you pass the actual prior messages as that key. The template splices them in between the system message and the new question.

Expected output (the placeholder expanded into the conversation):

```
[SystemMessage('You are a helpful assistant.'),
 HumanMessage('My name is Sam.'),
 AIMessage('Nice to meet you, Sam!'),
 HumanMessage('What's my name?')]
```

WHY THIS MATTERS LATER

You're seeing the mechanism behind **conversation memory**. A chatbot keeps a growing list of past messages and feeds it into `history` on every turn so the model "remembers." In **Phase 5** we'll automate keeping that list using LangGraph (the modern approach) instead of managing it by hand — but the placeholder is the foundation.

3.4 — Few-shot prompting: teaching by example

TERMS: ZERO-SHOT VS FEW-SHOT

Zero-shot = you just ask, giving no examples (everything so far). **Few-shot** = you include a handful of input → output *examples* in the prompt so the model copies the pattern. Few-shot is the cheapest way to steer format, tone, or tricky behaviour without any training.

You could paste examples into a string by hand, but `FewShotChatMessagePromptTemplate` does it cleanly: you give it a list of example dicts and a template describing how to render *one* example as messages.

```
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import (
    ChatPromptTemplate, FewShotChatMessagePromptTemplate,
)
from langchain_core.output_parsers import StrOutputParser

load_dotenv()

examples = [                                     # 1
    {"word": "happy", "emoji": " "},
    {"word": "fire", "emoji": " "},
    {"word": "ocean", "emoji": " "},
]

example_prompt = ChatPromptTemplate.from_messages([    # 2 how ONE example looks
    ("human", "{word}"),
    ("ai", "{emoji}"),
])

few_shot = FewShotChatMessagePromptTemplate(
    # 3 stamp the template over every example
    examples=examples,
    example_prompt=example_prompt,
)

final_prompt = ChatPromptTemplate.from_messages([    # 4 assemble the full
    prompt
    ("system", "You translate a single word into one emoji."),
    few_shot,
    # the examples get inserted here
    ("human", "{word}"),                                # the real question
])

chain = final_prompt | ChatGoogleGenerativeAI(model="gemini-2.5-flash") |
StrOutputParser()
print(chain.invoke({"word": "star"}))
```

Line	Explanation
1	Your examples as a list of dicts. Keys (<code>word</code> , <code>emoji</code>) match the blanks in the example template.
2	A mini-template describing how a <i>single</i> example becomes messages: a human turn (the word) and an ai turn (the emoji).

3	<code>FewShotChatMessagePromptTemplate</code> applies that mini-template to every example, producing a block of demo conversation turns.
4	The final prompt = system rules + the few-shot block + the real question. The model sees three worked examples, then answers in the same style.

Expected output:



WHY FEW-SHOT WORKS

The model treats the examples as a pattern to continue. Three "word → emoji" pairs make "give me exactly one emoji, nothing else" obvious — far more reliable than describing the rule in words. Use few-shot whenever a model *almost* does what you want but drifts on format or style. Ollama swap is the usual one-liner.

3.5 — Output parsers & the problem with prose

You've used `StrOutputParser` since Phase 1: it pulls the plain string out of the model's reply. That's great when you want text. But often you want **data** your program can use — a number, a list, a record with fields. Ask a model for that in prose and you get unreliable mush:

```
# Fragile: parsing prose by hand
text = model.invoke("Give me a movie title and its year.").content
# -> "Sure! How about 'Inception' (2010)?" ...now good luck extracting 2010 reliably
```

Output parsers turn replies into structured Python. The two you'll meet most:

Parser	Returns	When
<code>StrOutputParser</code>	a plain <code>str</code>	You just want the text.
<code>JsonOutputParser</code>	a Python <code>dict</code> / <code>list</code>	You asked the model for JSON and want it parsed.

But the most robust modern approach skips manual parsing entirely — you declare the shape you want and let LangChain enforce it. That's the next section.

3.6 — Structured output with Pydantic (the modern way)

TERMS: SCHEMA, PYDANTIC, STRUCTURED OUTPUT

A **schema** is a description of the shape of data — which fields exist and their types. **Pydantic** is the standard Python library for declaring schemas as classes. **Structured output** is asking the model to return data that fits a schema, validated automatically — no fragile text parsing.

You define the shape as a Pydantic class, then call `model.with_structured_output(YourClass)`. The model is constrained to return exactly that shape, already parsed into a Python object:

```
from dotenv import load_dotenv
from langchain_google_genai import ChatGoogleGenerativeAI
from pydantic import BaseModel, Field # 1

load_dotenv()

class Movie(BaseModel): # 2 the shape we want back
    """A movie and its key facts."""
    title: str = Field(description="The film's title")
    year: int = Field(description="Year of release")
    director: str = Field(description="The director's full name")
    rating: float = Field(description="IMDb-style rating out of 10")

model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")
structured = model.with_structured_output(Movie) # 3 constrain the model to Movie

result = structured.invoke("Tell me about the movie Inception.") # 4

print(type(result)) # -> a real object, not text
print(result.title) # Inception
print(result.year) # 2010
print(result.director) # Christopher Nolan
```

Line	Explanation
1	Import <code>BaseModel</code> (the base class for schemas) and <code>Field</code> (lets you attach a description to each field — the model reads these as hints).
2	Declare the shape: four fields with Python types (<code>str</code> , <code>int</code> , <code>float</code>). The docstring and field descriptions guide the model on what to fill in.
3	<code>with_structured_output(Movie)</code> returns a new Runnable that <i>must</i> answer in the <code>Movie</code> shape. Under the hood LangChain uses the model's tool/JSON features to enforce it.
4	<code>invoke</code> returns a fully-formed, validated <code>Movie</code> object — access fields with dot notation. No <code>.content</code> , no parsing, no regex.

Expected output:

```
<class '__main__.Movie'>
Inception
2010
Christopher Nolan
```

OLLAMA TOO — WITH A CAVEAT

`with_structured_output` works on local models that support tool/function calling — `llama3.2` does. Swap in `ChatOllama(model="llama3.2")` and the same code works. If a small local model struggles to

fill the schema reliably, either pick a more capable model or add clearer `Field` descriptions. Validation (the Pydantic types) will catch a malformed answer rather than letting bad data through silently.

RULE OF THUMB

Need text? End your chain with `StrOutputParser()`. Need data your program will act on? Use `with_structured_output(YourPydanticModel)`. Reach for raw `JsonOutputParser` only when a provider can't do structured output natively.

3.7 — Common errors & troubleshooting

Symptom	Likely cause & fix
<code>KeyError</code> / "missing variable" when invoking a template	Your dict is missing a blank, or a key is misspelled. Every <code>{name}</code> in the template needs a matching dict key.
Curly braces in your text cause errors (e.g. JSON examples)	Templates treat <code>{ }</code> as variables. To include a literal brace, double it: <code>{{ }</code> and <code>}}</code> .
<code>MessagesPlaceholder</code> errors with "missing"	You didn't pass its key. Either always pass the list, or make it optional: <code>MessagesPlaceholder("history", optional=True)</code> .
Few-shot: example keys don't appear	The keys in your <code>examples</code> dicts must match the blanks in <code>example_prompt</code> exactly.
Structured output: <code>NotImplementedError</code> or empty/garbled result	The chosen model doesn't support structured output, or is too small. Use a tool-calling-capable model (Gemini, <code>llama3.2</code>) and add <code>Field</code> descriptions.
Pydantic <code>ValidationError</code>	The model returned data that doesn't fit your types (e.g. text where an <code>int</code> was required). Loosen the type, clarify the field description, or retry.

3.8 — Mini practice exercise

YOUR TURN (DO THIS BEFORE PHASE 4)

Task: Build a "review analyser." Define a Pydantic schema `Review` with fields `sentiment` (a string: positive / neutral / negative), `score` (an int 1–5), and `summary` (a one-line string). Use `with_structured_output` so that invoking with a customer review string returns a `Review` object. Then add a **few-shot** twist: give two example reviews-with-labels in the prompt to steer the style of the summary. Run it on both Gemini and Ollama.

Why: it fuses the whole phase — templates, few-shot, and Pydantic structured output — into the kind of "text in, clean data out" component you'll reuse constantly (and which feeds neatly into the chains of Phase 4).

Hints (not the full answer):

- Start simple: get `with_structured_output(Review)` working with a plain string question *before* adding few-shot.

- For the 1–5 score, `score: int` with a clear `Field(description="1 (worst) to 5 (best)")` guides the model.
- To combine few-shot with structured output, build the `final_prompt` (§3.4), then pipe it into the structured model: `chain = final_prompt | model.with_structured_output(Review)`.
- If Ollama returns shaky data, add sharper field descriptions or try a clearer system instruction — the Pydantic validation will tell you exactly what didn't fit.

Terms introduced this phase

Term	One-line meaning
Prompt template	Reusable prompt with named <code>{blanks}</code> filled from a dict; a Runnable.
ChatPromptTemplate	Builds a list of role-tagged messages from <code>(role, text)</code> tuples.
ChatPromptValue	What a chat template produces: a wrapped list of messages.
MessagesPlaceholder	A template slot filled at runtime with a <i>list</i> of messages (e.g. history).
Zero-shot	Asking with no examples.
Few-shot	Including example input → output pairs so the model copies the pattern.
FewShotChatMessagePromptTemplate	Renders a list of examples into demo conversation turns.
Output parser	A Runnable that converts a model reply into a usable Python value.
Schema	A description of data's shape: which fields, which types.
Pydantic / BaseModel	The standard library for declaring schemas as Python classes.
Structured output	Model output constrained to a schema and auto-validated, via <code>with_structured_output</code> .

PHASE 3 COMPLETE

You can now craft reusable prompts with roles and variables, splice in dynamic message history, teach a model by example with few-shot, and — most importantly — get back clean, validated data with Pydantic structured output instead of brittle prose. **Next — Phase 4: Chains & LCEL Composition**, where these pieces stop being one-line pipes and become real pipelines: running steps in *parallel*, *routing* between different sub-chains, dropping in custom Python functions, and debugging the data flowing through it all.